



Sliding Windows

- Gaurish Baliga

- * Fixed size sliding windows
- ↳ * Dynamic size sliding windows

Length = 3 $\rightarrow k$

$\frac{n}{k}$

1 3 4 1 7 2 1

1 3 4 1

8 8 12 10 10

$O(n^2)$

$$n \leq 2 \times 10^5 \quad \text{TLE}$$

↪ 3

$$\text{sum} = 8$$

$$8 - 1 + 1 = 8$$

X

	↪ 2				
1	3	4	1	7	2
—	—	—	—	—	—
	—	1	+		

$$8 - 3 + 7 = 12$$

$O(n)$

↳ data required
for all sliding
windows



Sliding Window

- Useful for array based problems with constant subarray size
- Optimization Technique
- When to use?
 - Calculating some information for every fixed length subarray in an array
- Use of 2 pointers.
- Useful for interviews too 😊

Problem 1

Big Hint → sliding window



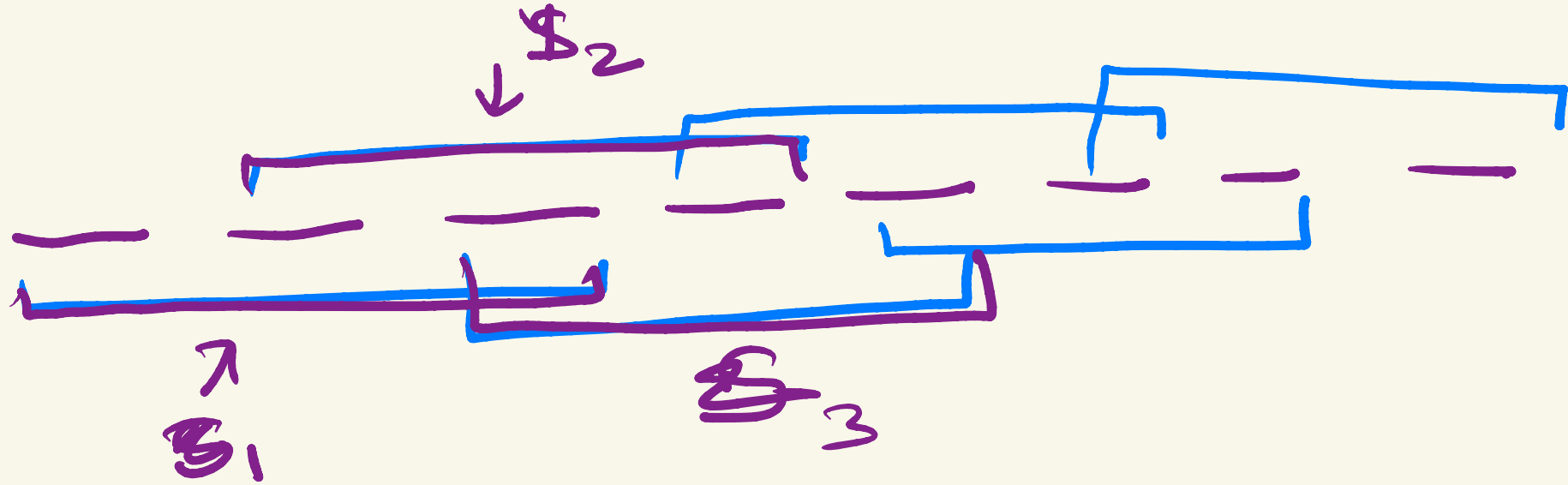
Find the subarray of size K which has the maximum sum in an array.

Input : arr = {1, 4, 2, 10, 23, 3, 1, 0, 20}, k = 4

Output : 39

Explanation: We get maximum sum by adding subarray {4, 2, 10, 23} of size 4.

* → We can iterate → $O(n)$ time
→ $\max(s_1, s_2, s_3, \dots)$



Approach



We make a window of size K and keep shifting it from left to right.

Adding a new element and removing an old element from the window must be done efficiently.

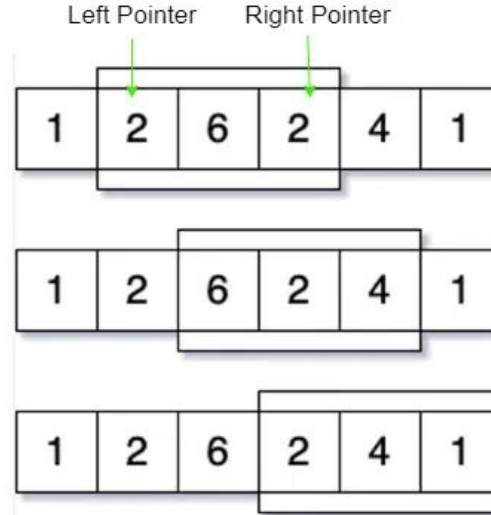
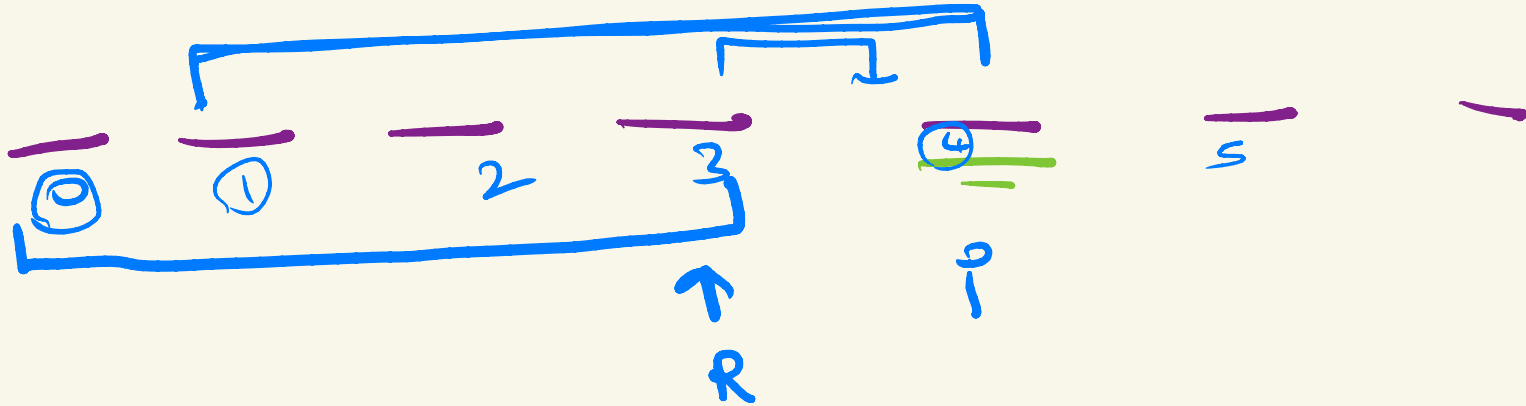


Image Credits - geeksforgeeks

*

$$\text{sum} = x + \text{arr}_i - \text{arr}_{i-k}$$

↳ ans = max(ans, sum)



Code

T.C.
 $O(n)$

Decide what information you want to store for a window

Create first window of size K and initialize the answer using the information in first window.

Slide the window by adding a new element and removing the first element and update the answer in each iteration.

```
1 // Given an array of integers, find the maximum sum of subarray of size k.
2 // Time complexity: O(n)
3 // Space complexity: O(1)
4 int maxSubarraySumOfSizeK(vector<int>& a, int k) {
5     int n = a.size();
6
7     // the info we need to store for all subarrays of size k
8     int sum = 0;
9
10    // add the first k element to the window
11    for (int i = 0; i < k; i++) {
12        sum += a[i];
13    }
14
15    // initialize answer with the info of the first window
16    * int ans = sum;
17
18    // slide the window
19    for (int i = k; i < n; i++) { // slide the window
20        // add the new element to the window
21        + sum += a[i];
22
23        // remove the first element from the window
24        - sum -= a[i - k];
25
26        // update the answer
27        ans = max(ans, sum);
28    }
29
30    return ans;
31 }
```

first K elmts

Right pointers

Code

Merged the for loops to make the code less repetitive and shorter

```
1  int maxSubarraySumOfSizeK(vector<int>& a, int k) {
2      int n = a.size();
3
4      // the info we need to store for all subarrays of size k
5      int sum = 0, ans = 0;
6
7      // merged both loops into one loop
8      → for (int i = 0; i < n; i++) {
9          // add the new element to the window
10         sum += a[i];
11
12         // remove the first element from the window if i ≥ k
13         if (i ≥ k) {
14             sum -= a[i - k];
15         }
16
17         // update the answer if i ≥ k - 1
18         if (i ≥ k - 1) {
19             ans = max(ans, sum);
20         }
21     }
22
23     return ans;
24 }
```

Handwritten red notes:
2nd subarray [(lines 12-15)
size k is formed [(lines 18-21)

Problem 2



Find the subarray of size K which has the maximum distinct elements in an array.

Input : arr = {1, 1, 2, 1, 3, 2, 1, 1, 2}, k = 4

Output : 3

Explanation: Subarray {1, 3, 2, 1} of size 4 has 3 unique elements.

Set

→ Handling duplicates in an issue!

Ans = 2

$$\begin{array}{ccccc} 2 & 2 & 2 & 2 & 1 \\ \hline \uparrow & \underbrace{\hspace{10em}} \end{array}$$
$$\begin{aligned} & \{2, 3\} \\ \hookrightarrow & \{2, 1, 3\} \\ \hookrightarrow & \{1, 3\} \end{aligned}$$

Map

TC $\rightarrow$ $O(n \log k)$

$\rightarrow$ $mp.size()$ $\rightarrow$ number of keys in the map

$\rightarrow$ number of distinct elements

*
 $mp[arr_i]++$
 $mp[arr_i - k]--$

$mp.erase(arr_i - k)$

1 2 3 4

2 : 1
3 : 1
4 : 1

unordered_map not preferred in
cpl. / hashmap

↳ Collisions

near wu



operation

$O(\log(\underline{|\text{map}|}))$

↑
atmost k elements

Problem 3

→ Implement ??



Find the subarray of size K which has the maximum sum and all distinct elements.

→ all elements should be distinct
→ sum should be as large as possible

Input : arr = {1, 1, 2, 1, 3, 2, 1, 1, 2}, k = 3

Output : 6

Explanation: Subarray {1, 3, 2} of size 3 has all distinct elements and maximum sum.

KEYS
→
{ 2, 2 }
{ 1, 1 }
{ 3, 1 }

→ maximum sum such that $O(n \log k)$

1. $size = k$

2. all elements are distinct

↳ $map.size() = k$

k Distinct Elements

→ $map<int, int> mp$

→ sum

answer

only updated on

condition

Task

For all problems today,
write code that returns
index/entire subarray

$O(n \log k) \rightarrow O(n)$ Queue/Deque Optimizations



- The elements which are added first in the window will be removed first, so sometimes we can optimize our codes by using queue / deque instead of sets.
- Example -> if we have to find the first element of a particular type in each subarray, we can store the indices of all elements of that type in a queue and the index of first such element in a window will be present at the front of queue for all subarrays.



Problem 5

For each subarray of size K in array, find the index of the **minimum element** in that subarray.

↳ print index of leftmost min element

Input : arr = {^{1 2}1, 3, 1, 2, 4, 5, 1, 7, 2}, k = 3

Output : 1 3 3 4 7 7 7
 ↑ ↑

min-heap → {value, index}

Naive approach

$O(n \log k)$ $\{1, 2\}$ $\{2, 1\}$
 $\{1, 3\}$

set < pair<int, int>>

1. What does set sort upon??

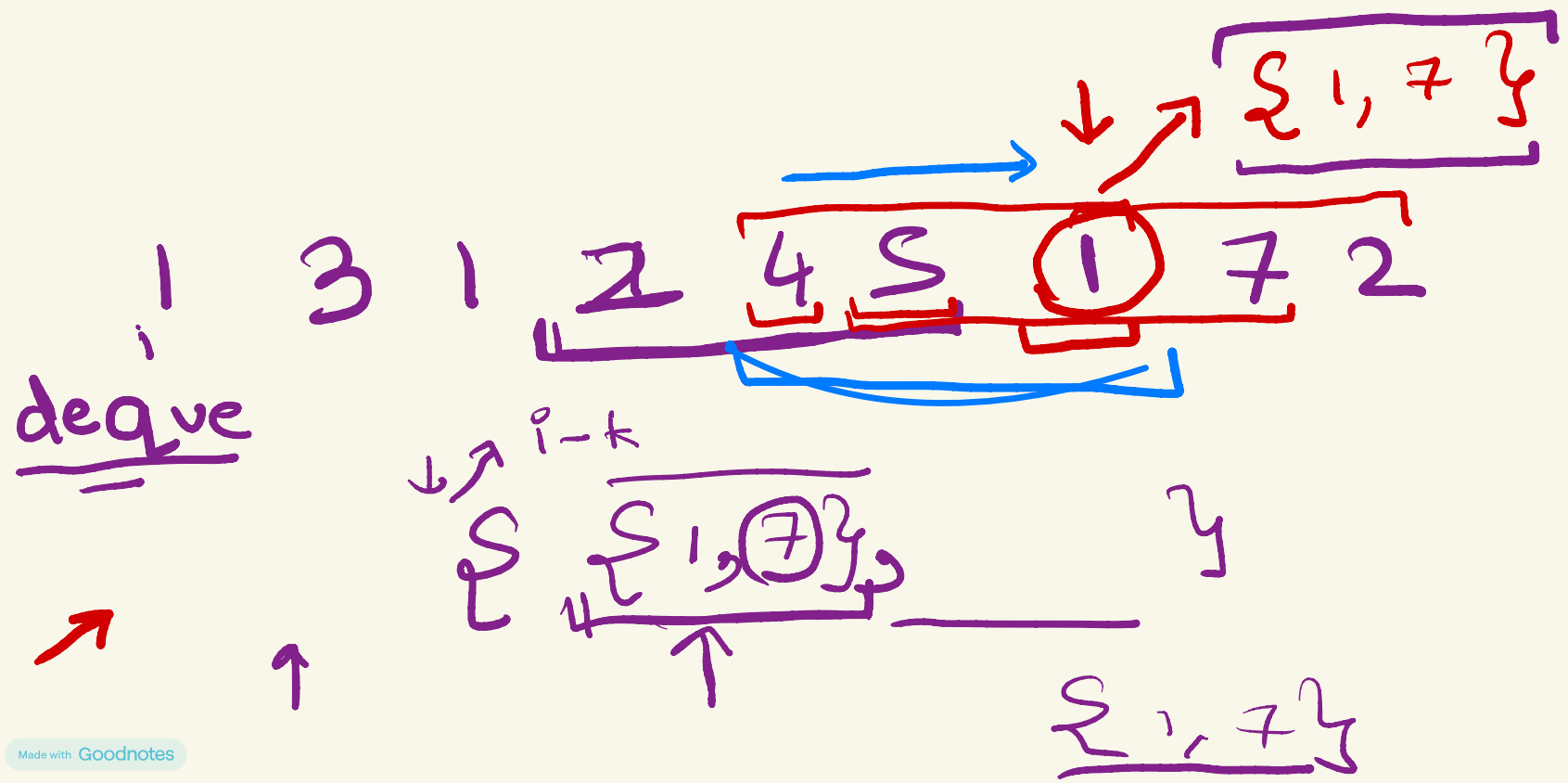
$\{1, 2\}$, $\{1, 3\}$, $\{2, 1\}$ e
↑ ↑ ↑
* nice trick

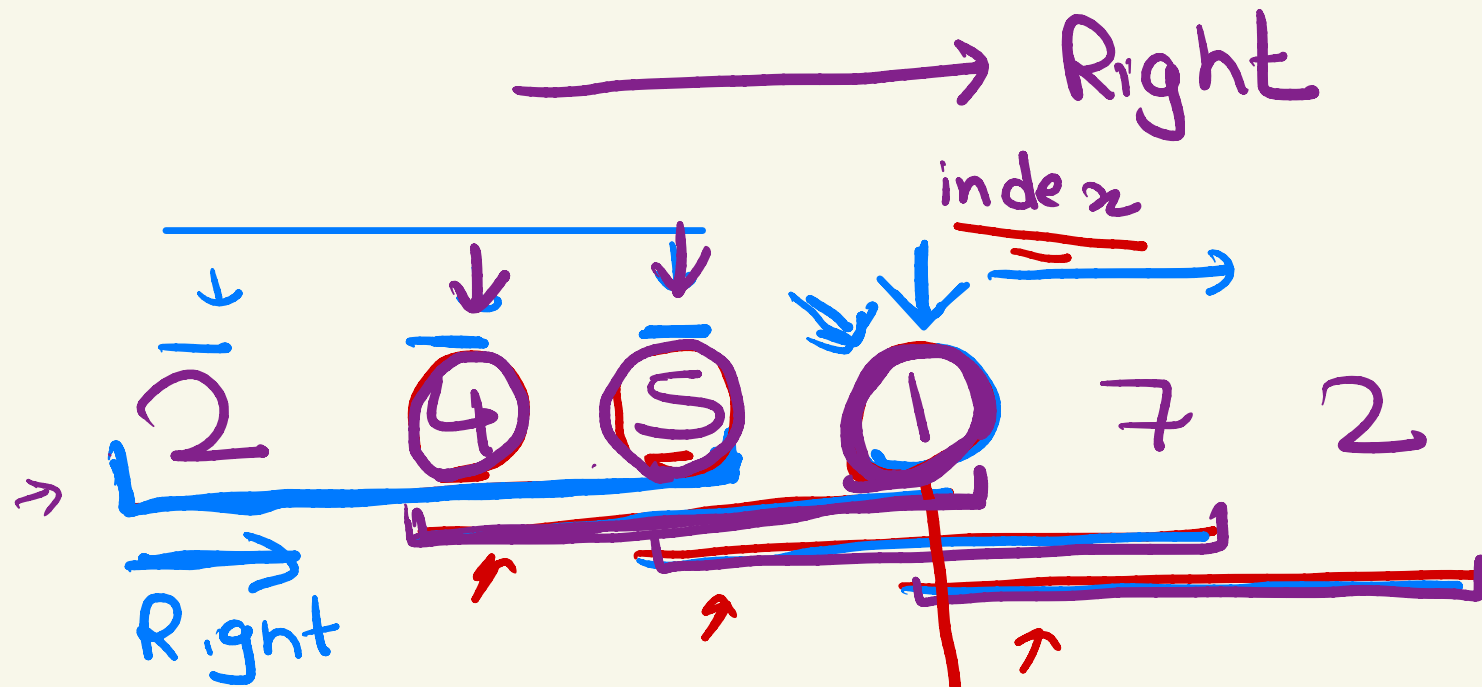
2. leftmost min index ??

$\{arr[i], i\}$

* s.begin() → {min value, left most}

$O(n \log k)$ $\longrightarrow$ $O(n)$ * important





consider all subarrays of size k containing I

1, 1, 1, 1, 1, ...
1 2 3 4 5, ...

$k = 3$

Solution

Sliding Window
Minimum in O(N)

pop
technique [
out
of window

```
1  vector<int> sliding_window_minimum(vector<int> &a, int k) {  
2      int n = a.size();  
3      vector<int> ans;  
4      deque<int> deq;  
5  
6      for (int i = 0; i < n; i++) {  
7          while (!deq.empty() && a[deq.back()] ≥ a[i])  
8              deq.pop_back();  
9          deq.push_back(i);  
10  
11         if (deq.front() == i - k)  
12             deq.pop_front();  
13  
14         if (i ≥ k - 1)  
15             ans.push_back(a[deq.front()]);  
16     }  
17  
18     return ans;  
19 }
```

queue $\rightarrow$ 1 insert / elements
 $\hookrightarrow$ 1 delete / element

2n ops on queue

TC $\rightarrow O(n)$

5 minutes

Problem 4



[For each subarray of size K in array, find the index of the first negative element in that subarray.

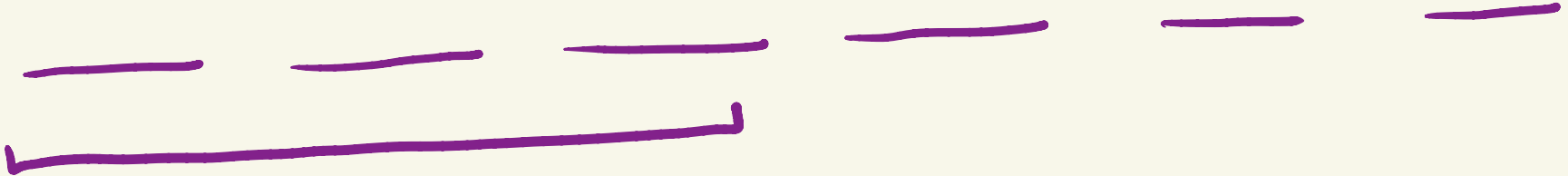
Input : arr = {1, 1, 2, -1, 3, 2, -3, -4, 2}, k = 3

Output : -1 4 4 4 7 7 7

-3 2 -5 7

queue <int> q;

i
 $a_i < 0$ $q.push(i);$



q store 2?

all indices of negative elements
in that window

$q.front() = i - k \rightarrow q.pop();$

if (q.size()) cout << q.front()
else -1



Some Resources

- Monotonic Stack & Queues:

cp-algorithms.com/data_structures/stack_queue_modification.html

- Good to know

- Sliding Window (USACO):

<https://usaco.guide/gold/sliding-window?lang=cpp>

- Has a bunch of cool sliding window problems